School of Science, Computing and Emerging Technologies

COS80023 Big Data

Pass Task 9: Natural Language Processing

Overview

Using R, in this tutorial you will learn how to process text to obtain the document frequency matrix. You'll apply a simple classifier to the DFM.

Purpose

Understand the basic principles of working with text by transforming the text so that it can be processed by a computer. Learn how classifiers can be applied to text data after transformation.

Task

Go through the steps described below. Answer the questions in a separate file.

Time

This task should be completed in your 9th tutorial or the week after and submitted to Canvas for feedback. It should be discussed and signed off in tutorial 9 or 10.

This task should take no more than 2 hours to complete (excluding introductory videos).

Resources

- Lecture material
- Code listing (r script) with more explanations
- Any other material you find useful in explaining the results
- You can use genAI for your research or additional clarification

Feedback

Demonstrate your steps and discuss your answers with the tutorial instructor.

Next

This is the last of your pass tasks. Continue with credit tasks if you like.

Pass Task 9 — Submission Details and Assessment Criteria

Write down the questions and answers in a text or Word document, convert to pdf and upload to Canvas. Your tutor will mark the submission on line. If the submission is not marked as '1', it is considered as incomplete and must be resubmitted.

Task 9

You will be working with large tables. To visualise the outcome use

`mytable[1:20, 1:100] or View(mytable[1:20, 1:100])`

to show lines 1 – 20 and columns 1 – 100 (change the numbers as you like). Displaying the whole table can take several minutes.

`ncol(mytable)` and `nrow(mytable)`

tell you how many columns or rows your table has.

Exercise 1

Run these lines in your RStudio. They load a text dataset "sms"-with annotations (labels), then process the text part of the dataset. The dataset contains text messages (SMS) and a label that tells us whether the text message is ham (a legitimate text message) or spam (an unwanted message trying to sell something).

```
library("caret")
library("quanteda")
library(ggplot2)
data.raw <- read.csv(file.choose(), stringsAsFactors = FALSE,
fileEncoding = "UTF-8")
data.raw$type <- as.factor(data.raw$type)
prop.table(table(data.raw$type))
```

Question 1. What do these numbers tell you? Why did we give the table() function the argument data.raw$type?

Exercise 2

Run the following lines and examine the outcome.

```
indexes <- which(data.raw$type=="ham")
ham <- data.raw[indexes,]
spam <- data.raw[-indexes,]
spam$TextLength <- nchar(spam$text)
ham$TextLength <- nchar(ham$text)
summary(ham$TextLength)
summary(spam$TextLength)
ggplot(data.raw, aes(x = TextLength, fill = type)) +
  theme_bw() +
  geom_histogram(binwidth = 5) +
  labs(y = "Text Count", x = "Length of Text",
       title = "Distribution of Text Lengths with Class Labels")
```

Question 2. What does this code do, and what does the output tell us about the properties of ham and spam text messages?

Exercise 3

Run the following lines in R:

```
set.seed(32984)
indexes <- createDataPartition(data.raw$type, times = 1,
                               p = 0.7, list = FALSE)

train.raw <- data.raw[indexes,]
test.raw <- data.raw[-indexes,]

train.tokens <- tokens(train.raw$text, what = "word", remove_numbers
                = TRUE, remove_punct = TRUE, remove_symbols = TRUE,
                remove_hyphens = TRUE, ngrams = 1)
train.tokens <- tokens_tolower(train.tokens)
```

Quanteda's list of stop words is

```
stopwords()

train.tokens <- tokens_select(train.tokens, stopwords(), selection =
                "remove")
train.tokens <- tokens_wordstem(train.tokens, language = "english")
train.tokens.dfm <- dfm(train.tokens, tolower = FALSE)
```

Question 3. Which of the NLP processing steps you learned are performed by this code? Is the outcome what you expect?

Question 4. How do you create 3-grams? (Document the line of code needed.) How does the content of data.tokens change?

Exercise 4

Now that you have prepared the data, you can create the document frequency matrix and apply the classifier. This dataset is large, so rather than using SVM, we apply a simple decision tree.

To be able to add the label (annotation), you first have to change the matrix to a frame:

```
train.tokens.df <- convert(train.tokens.dfm, to = "data.frame")
train.tokens.df <- cbind(type = train.raw$type, data.tokens.df)
```

Question 5. What has cbind changed in the data?

Clean up column names (removes invalid tokens like . at the start of the word)

```
names(train.tokens.df) <- make.names(names(train.tokens.df))

train.tokens.df <- train.tokens.df[,
!duplicated(colnames(train.tokens.df))]
```

Create settings for training, using stratified samples as before

```
set.seed(48743)

folds <- createMultiFolds(train.tokens.df$type, k = 10, times = 1)

traincntrl <- trainControl(method = "repeatedcv", number = 10,
repeats = 3, index = folds)
```

This is a large dataset, so we use a simple decision tree as classifier

```
rpart_model <- train(type ~ ., data = train.tokens.df, method =
"rpart", trControl = traincntrl, tuneLength = 7)
```

Observe the accuracy on the training set.

Now you have to prepare the test set like the training set:

```
test.tokens <- tokens(test.raw$text, what = "word", remove_numbers =
               TRUE, remove_punct = TRUE, remove_symbols = TRUE,
               remove_hyphens = TRUE, ngrams = 1)


test.tokens <- tokens_tolower(test.tokens)

test.tokens <- tokens_select(test.tokens, stopwords(), selection =
               "remove")

test.tokens <- tokens_wordstem(test.tokens, language = "english")

test.tokens.dfm <- dfm(test.tokens, tolower = FALSE)
```

Here you adjust the columns of the test set to the columns of the training set:

```
test.tokens.dfm <- dfm_select(test.tokens.dfm, pattern =
train.tokens.dfm, selection = "keep")
```

Question 6. You did not do this in your first practice on classifiers. Why do you have to do it here?

```
test.tokens.df <- convert(test.tokens.dfm, to = "data.frame")

test.tokens.df <- cbind(type = test.raw$type, test.tokens.df)

names(test.tokens.df) <- make.names(names(test.tokens.df))

test.tokens.df <- test.tokens.df[, !duplicated(colnames(test.tokens.df))]
```

To compare the complete training and testing results, you can apply the model to both the training and testing data sets:

```
trainresult <- predict(rpart_model, newdata=train.tokens.df)

testresult <- predict(rpart_model, newdata=test.tokens.df)

confusionMatrix(table(trainresult, train.tokens.df$type))

confusionMatrix(table(testresult, test.tokens.df$type))
```

Question 6. How does the model perform on the training and testing sets in terms of accuracy, specificity and sensitivity?